

Leveraging Target Domain Quirks for Four RCEs on a Newly Launched SRC

By J0o1ey · Translated from the Chinese original

0x01 Preface

Friends interested in technical exchange, or in pentest / code audit / SRC bug hunting / red team training, red-vs-blue assessments, security product development, or security services, are welcome to reach out via QQ/WeChat: 547006660.

<https://github.com/J0o1ey/BountyHunterInChina>

If you enjoy the "Reborn as a Bounty Hunter" series, please give it a star.

0x02 The Origin

One morning I was idly scrolling WeChat group chats when I noticed a newly launched SRC program.



Newly launched SRCs are usually tender rookies, untouched by the hordes — much easier to score. Let's take a look.

0x03 Asset Collection to a Changed Default Key — Dead End

I did a quick cyberspace asset sweep using the target's certificate information.

Quite a few of the target's domains followed a "hyphen naming" style. Big vendors often use this convention to distinguish development, testing, and production environments.



	-cdn-uat.:	183.
15	r-dev.ze	121.
15		183.
15		121.
15		59.1
159	-gateway-test.:	121.
161	pr-m	121.
162	n-dev.	121.

uat
test
pr
dev
域名关键词频出

In summary, the common keywords in dev/test/production domain names are:

```
uat
test
dev
pre
pr
pro
...
```

From the asset list I picked out a domain that looked like a management API endpoint and visited it.

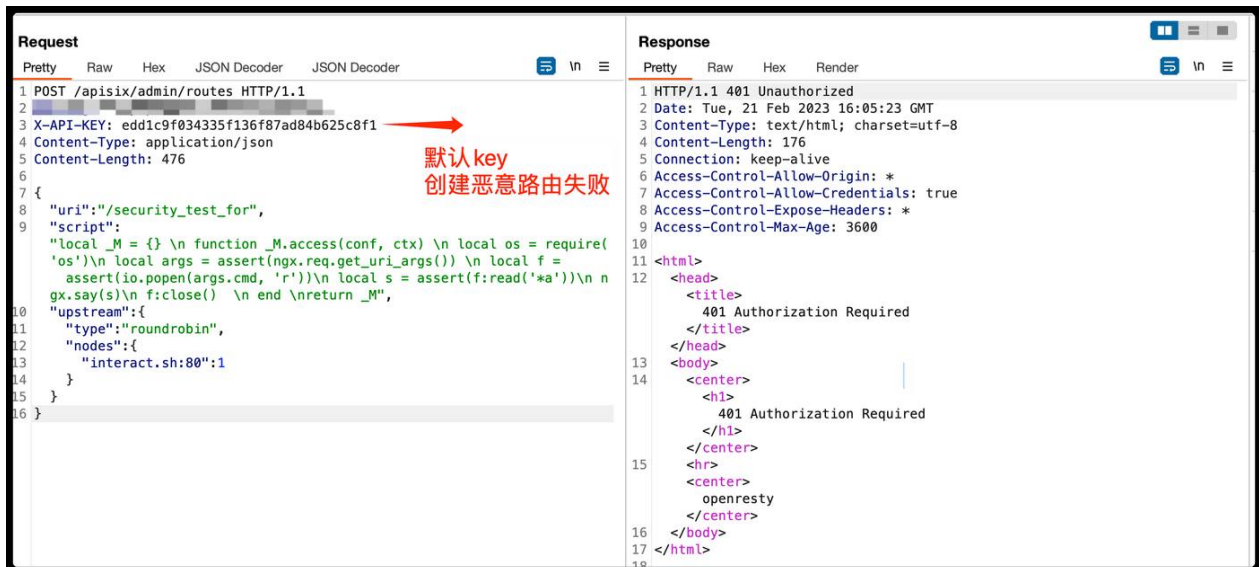


1	// 20230222000123
2	// https://gateway-!
3	
4	{
5	"error_msg": "404 Route Not Found"
6	}

Based on the page response, combined with years of testing experience, I concluded it ran Apache APISIX.

I had previously reproduced the APISIX default-key RCE — adding a malicious route with the default key — so let's try it right here.

Instant death: the production API had changed the default key, so no malicious routes could be created.



Is it over just like that? That's clearly not our style.

0x04 Reframing — Finding Hidden Test-Environment Assets

During asset collection we already identified the target's uniform naming convention.

So we can FUZZ the domains to strike some sparks and dig out hidden assets.

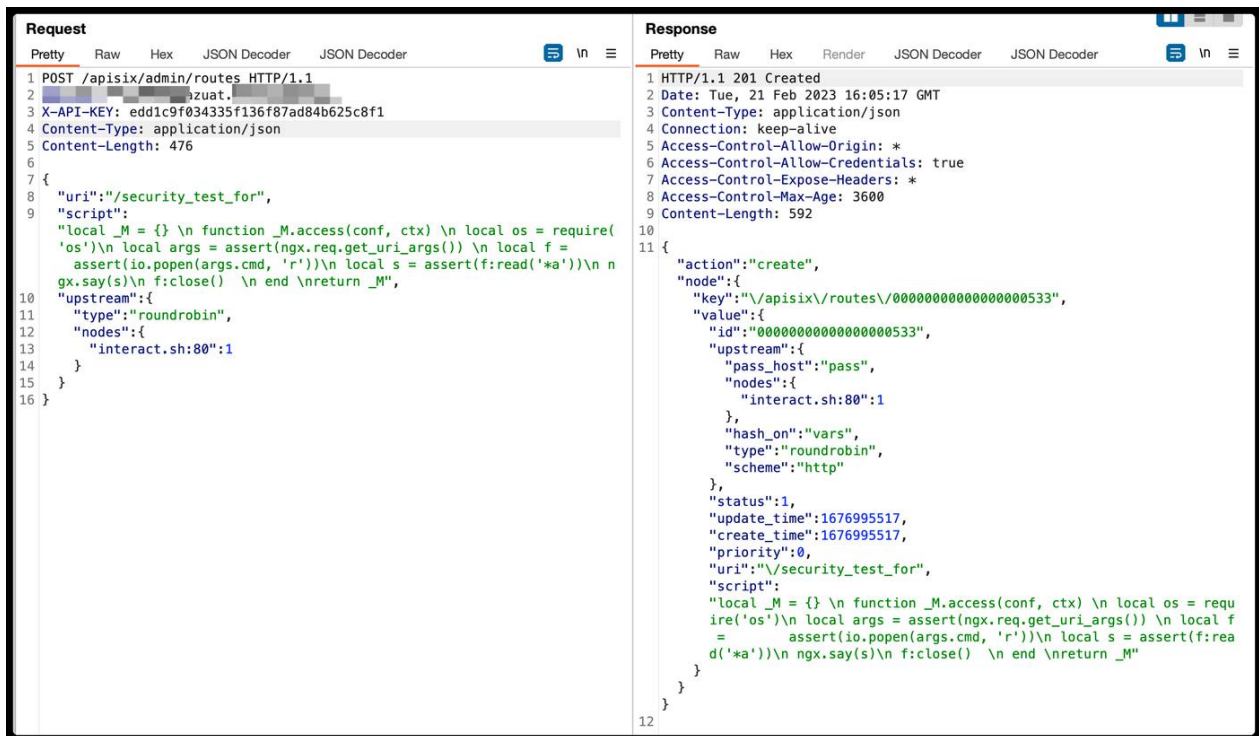


```
./ffuf -w domain_test -u https://gateway-xxx-xxx-FUZZ.xxx.com -mc 404 -t 1
```

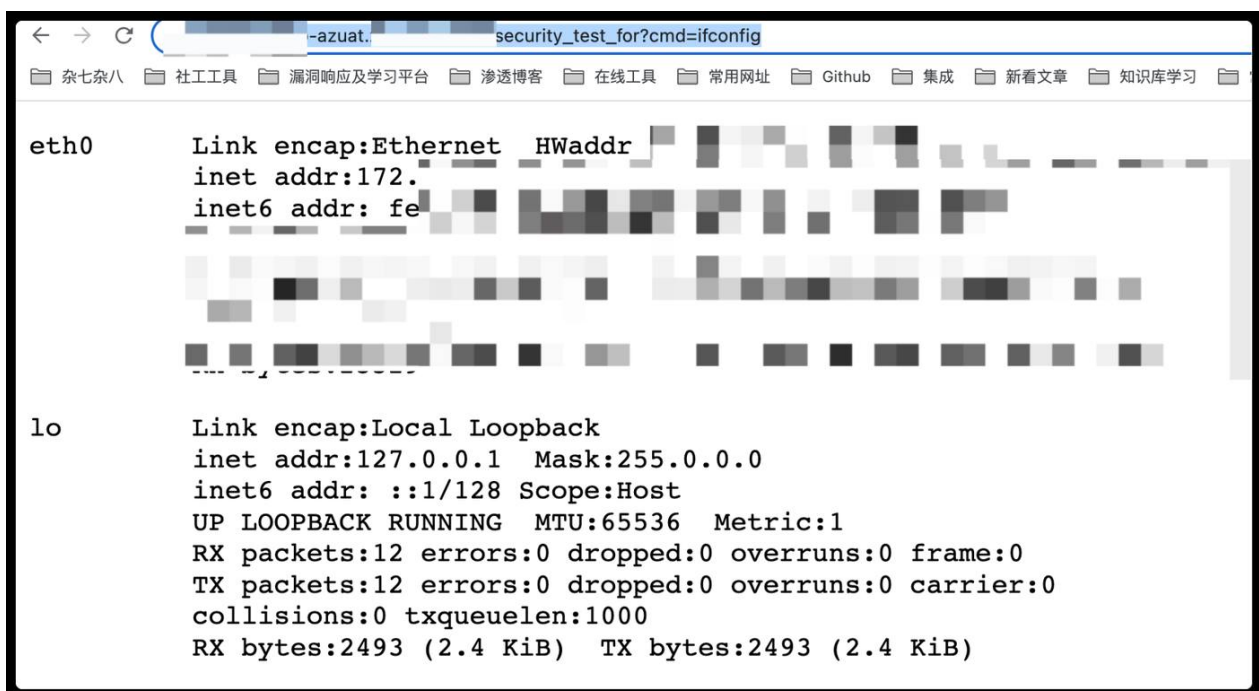
Eventually we discovered four hidden non-production assets following the target's naming scheme.

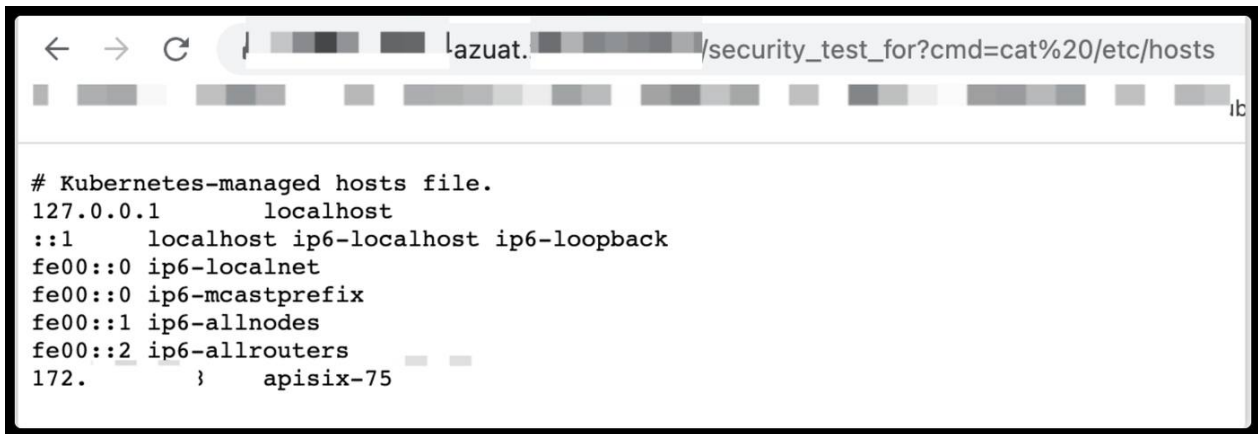
0x05 The Original Sin of Default Keys in Test Environments, to RCE

I then tried adding a malicious Lua route with the default key on the four hidden subdomains — all succeeded.



With the malicious routes in place, it was open plains all the way to the throne room.



A screenshot of a web browser window. The address bar shows a URL with a domain that has been redacted with grey boxes, followed by "/security_test_for?cmd=cat%20/etc/hosts". The browser's tab bar shows a single tab with a redacted title. The main content area displays the output of the command, which is the content of the /etc/hosts file. The output is as follows:

```
# Kubernetes-managed hosts file.
127.0.0.1    localhost
::1        localhost ip6-localhost ip6-loopback
fe00::0    ip6-localnet
fe00::0    ip6-mcastprefix
fe00::1    ip6-allnodes
fe00::2    ip6-allrouters
172.17.0.1  apisix-75
```

The target ran on k8s. A quick guess: the test environment used an old image with the default key baked in, and ops never changed it — hence the RCE pot.

After reporting the four command-execution findings, I rewarded myself with an egg rice-noodle roll plus an extra sausage for dinner.

0x06 Technical Takeaways

Leverage the target's domain-naming patterns to discover hidden development and test environment assets —

and break new ground.